

The MLflow logo is centered on a white background. The background features abstract geometric shapes in light gray and white, creating a modern, minimalist aesthetic. The text "MLflow" is rendered in a bold, black, sans-serif font. The "M" and "L" are uppercase, while "flow" is lowercase. The overall design is clean and professional, typical of a corporate or technical presentation slide.

MLflow

- 실험 추적이 필요한 순간

> 정확도 숫자만으로는 실험을 복원하기 어렵다

- model_final.pkl, model_final2.pkl 중 어느 파일을 사용할까?
- 점수가 오른 이유가 파라미터인지 데이터 변경인지 알 수 있을까?
- 전처리와 패키지 버전이 달라도 같은 예측이 나올까?
- 한 번의 학습을 식별하고, 그때의 조건과 결과를 함께 남긴다

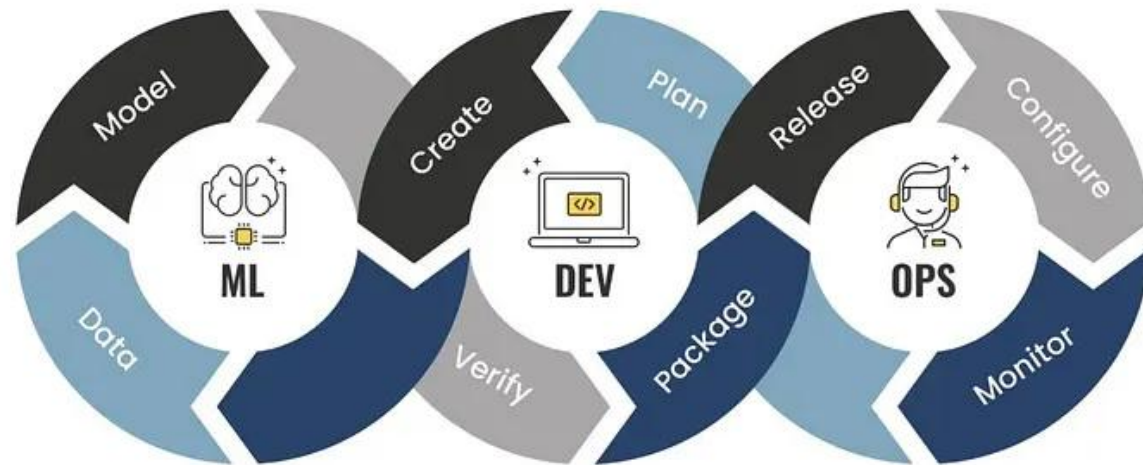
어제 실험
RandomForest, n_estimators=200
정확도 0.94 (변수 값은 기억 안 남)

오늘 다시 실행
똑같이 실행한 것 같은데
정확도 0.87 — 무엇이 달랐는지 알 수 없음

• MLOps의 범위

> 학습 이후에도 이어지는 모델의 생애주기

- 문제와 성공 기준을 정하고 데이터의 출처와 품질을 확인
- 학습 조건과 평가 결과를 기록하고 사용할 모델을 선택
- 선택 모델을 배포하고 실제 입력과 예측 결과를 관찰
- 성능 저하나 데이터 변화가 생기면 검토 후 다시 학습



- 왜 MLflow인가

> 이 수업에서는 MLflow를 사용합니다.

- 오픈소스이면서 사이킷런 등 대부분의 ML 라이브러리와 바로 연동되고,
- 개인 PC부터 회사 서버까지 어디에나 설치할 수 있어
- 가장 널리 쓰이는 실험 관리 도구입니다.

• MLflow 컴포넌트의 연결

> Tracking에서 남긴 모델을 Models와 Registry로 재사용

- Tracking: 실험 기록 및 조회: 코드, 데이터, 설정, 결과
- Models: 다양한 서빙 환경에 머신러닝 배포
- Projects: 어떤 플랫폼이든 재현 가능하게 실행할 수 있도록 패키징
- Registry: 저장소에 모델을 저장하고 관리



mlflow
Tracking

Record and query experiment:

- Code
- Data
- Config
- Results



mlflow
Projects

Package data science code in a format that enables reproducible runs on many platform



mlflow
Models

Deploy machine learning models in diverse serving environments



mlflow
Model Registry

Store, annotate and manage models in a central repository

- Experiment와 Run

- > 비교 목적과 한 번의 실행을 구분한다

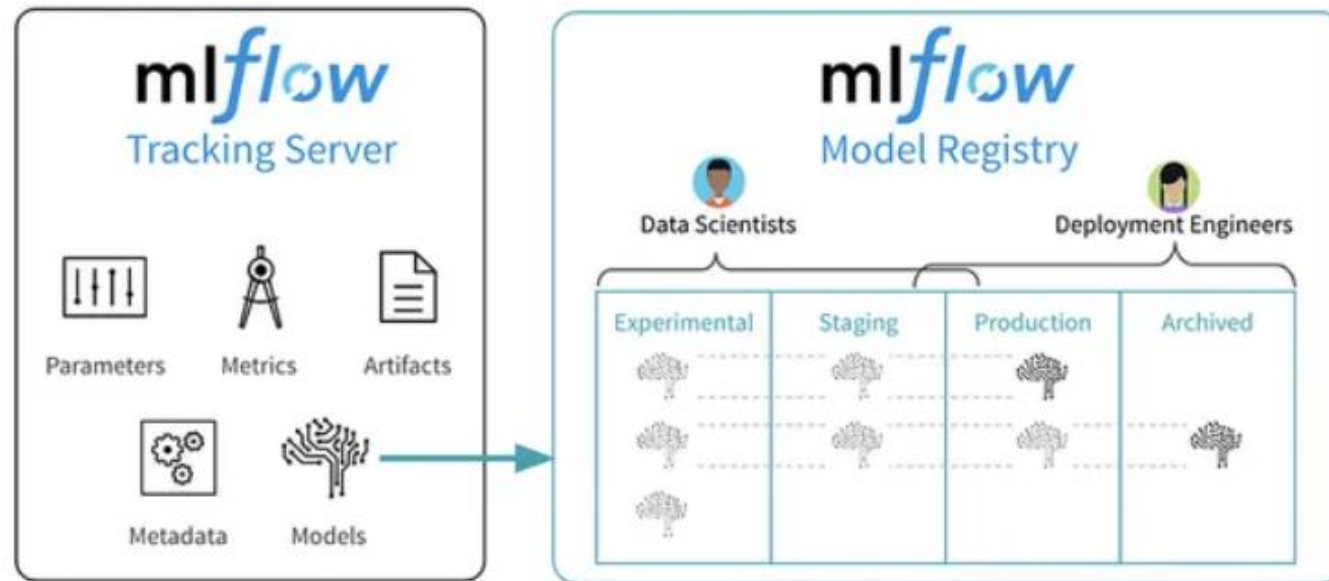
- Experiment: 같은 문제와 평가 기준을 공유하는 실행의 묶음
예시) Experiment: wine-classification
 - Run: 코드 실행 한 번의 기록 단위, 고유한 run_id로 식별
예시 Run: lr-C0.1, lr-C1, rf-depth4
 - Logged Model: Run에서 기록한 모델, MLflow 3에서는 model_id도 가짐

MLflow

• 기록할 정보의 종류

> 값의 의미에 맞는 기록 API

- Parameter: $C=1.0$, $\text{max_depth}=4$ 처럼 실행 조건을 기록
- Metric: $\text{val_f1_macro}=0.95$ 처럼 수치 결과와 step을 기록
- Tag: owner, purpose처럼 검색·설명용 문자열을 기록
- Artifact: 그림, JSON, CSV, 코드 등 파일을 저장
- Model: 예측 객체와 입력 규격·환경을 MLflow 형식으로 저장



• Tracking 저장 구조

> 브라우저·서버·저장소의 역할

- 학습 코드: Tracking URI를 통해 기록 API를 호출
- Tracking Server: 요청을 받고 UI와 기록 API를 제공
- Backend Store: Experiment, Run, parameter, metric, tag 저장
- Artifact Store: 모델 파일, 그림, 결과 파일 저장
- 이번 실습: 로컬 서버 + SQLite + 로컬 artifacts 폴더

- 실습 환경 준비

> Docker에서 파이썬 준비

```
python3 -m venv .venv  
python3 -m pip install -r requirements.txt
```

```
# requirements.txt  
mlflow  
scikit-learn  
pandas  
numpy  
matplotlib
```

• Tracking Server 실행

> 서버용 터미널을 열어 둔 채 실습을 진행한다

```
mlflow server `
  --host 127.0.0.1 --port 5000 `
  --backend-store-uri sqlite:///mlflow.db `
  --serve-artifacts --artifacts-destination ./artifacts
```

브라우저: <http://127.0.0.1:5000>

학습 코드에서는 항상 같은 주소 지정

```
import mlflow
```

```
mlflow.set_tracking_uri("http://127.0.0.1:5000")
```

완료 기준: UI 접속 성공. mlflow.db와 artifacts를 같은 수업 폴더에서 관리합니다.

MLflow

- MLflow 시작하기

- > 설치하기

- > pip으로 설치하고 버전을 확인합니다.

11

```
pip install mlflow
```

```
mlflow --version
```

Out : mlflow, version 3.16.1

- MLflow 시작하기

- > 실습: 설치 확인하기
- > 파이썬에서 바로 불러와 확인해 봅니다.

```
import mlflow  
print(mlflow.__version__)
```

Out: 3.16.1

• Tracking Server 구성 방식

> 같은 기록 API를 서로 다른 저장 구조에 연결

- 개인 실습: SQLite와 로컬 파일, 같은 PC에서 서버 실행
- 팀 사용: 공유 서버와 DB, 모든 사용자가 접근할 Artifact Store
- 서버 경유: 학습 클라이언트가 서버를 통해 파일을 업로드
- 직접 저장: 클라이언트가 S3 등의 저장소에 직접 업로드
- 서버를 생략하고 저장소 URI를 직접 쓰는 개인 구성도 가능

- MLflow Tracking 핵심

- > mlflow.set_tracking_uri로 저장소 지정하기
- > 이 수업에서는 SQLite 파일 하나를 백엔드로 사용합니다.

14

```
import mlflow

mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("breast-cancer-basic-tracking")
```

Out : INFO mlflow.tracking.fluent: Experiment with name
'breast-cancer-basic-tracking' does not exist. Creating a new experiment.

- MLflow Tracking 핵심

- > Parameter 기록하기: log_param
- > 학습에 사용한 설정값(하이퍼파라미터)을 기록합니다.

15

```
mlflow.log_param("n_estimators", 100)
```

```
mlflow.log_param("max_depth", 5)
```

```
# 여러 개를 한 번에 기록하려면
```

```
mlflow.log_params({"n_estimators": 100, "max_depth": 5})
```

• MLflow Tracking 핵심

- > Metric 기록하기: `log_metric`
- > 정확도처럼 학습 결과로 나오는 성능 지표를 기록합니다.
 - `step`을 주면 에폭별 변화 추이도 기록할 수 있습니다.

```
mlflow.log_metric("accuracy", 0.9708)
```

```
mlflow.log_metric("f1_score", 0.9772)
```

```
# 에폭마다 기록하고 싶다면 step 지정
```

```
for epoch, loss in enumerate(loss_history):
```

```
    mlflow.log_metric("loss", loss, step=epoch)
```

- MLflow Tracking 핵심

- > Artifact 기록하기: log_artifact

- > 그림, 데이터 파일, 모델 파일처럼 '파일 그 자체'를 남기고 싶을 때 사용합니다.

17

```
import matplotlib.pyplot as plt

plt.plot(loss_history)
plt.savefig("loss_curve.png")
mlflow.log_artifact("loss_curve.png")

# 폴더째로 남기려면
mlflow.log_artifacts("outputs/")
```

- MLflow Tracking 핵심

- > 실습: 사이킷런 모델 실험 로깅하기 (1) - 데이터 준비
- > 유방암 진단 데이터셋을 불러와 학습·테스트 데이터로 나눕니다.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

cancer = load_breast_cancer()
X = cancer.data
y = cancer.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- MLflow Tracking 핵심

- > 실습: 사이킷런 모델 실험 로깅하기 (2) - 학습과 기록
- > with 블록 안에서 학습을 진행하며 파라미터와 지표를 함께 기록합니다.

19

```
with mlflow.start_run(run_name="rf-n50-d3"):
    model = RandomForestClassifier(n_estimators=50, max_depth=3)
    model.fit(X_train, y_train)
    acc = accuracy_score(y_test, model.predict(X_test))
    mlflow.log_params({"n_estimators": 50, "max_depth": 3})
    mlflow.log_metric("accuracy", acc)
```

Out : accuracy = 0.9708

- MLflow Tracking 핵심

- > Tag로 메타 정보 남기기
- > 담당자, 데이터 버전처럼 검색·필터링에 쓸 부가 정보는 태그로 남깁니다.

```
mlflow.set_tag("model_type", "RandomForest")  
mlflow.set_tag("dataset", "breast_cancer")  
mlflow.set_tag("owner", "juwan")
```

- MLflow Tracking 핵심

- > Autolog: 한 줄로 자동 기록하기
- > 사이킷런 모델은 autolog 한 줄이면 파라미터·지표·모델이 자동으로 기록됩니다.

21

```
mlflow.sklearn.autolog()

with mlflow.start_run(run_name="autolog-RandomForest"):
    model = RandomForestClassifier(n_estimators=150, max_depth=6)
    model.fit(X_train, y_train)
    # log_param/log_metric을 직접 호출하지 않아도 자동 기록됨
```

Out: training_accuracy_score: 1.0
training_f1_score: 1.0
training_log_loss: 0.1188

- MLflow Tracking 핵심

- > 실습: Autolog로 여러 모델 비교하기
- > 모델 종류만 바뀌가며 반복 실행해도 각 모델에 맞는 파라미터가 자동으로 기록됩니다.

22

```
models = {
    "RandomForest": RandomForestClassifier(n_estimators=150, max_depth=6),
    "LogisticRegression": LogisticRegression(max_iter=2000),
    "GradientBoosting": GradientBoostingClassifier(n_estimators=100),
}
for name, model in models.items():
    with mlflow.start_run(run_name=f"autolog-{name}"):
        model.fit(X_train, y_train)
        mlflow.log_metric("test_accuracy", model.score(X_test, y_test))
```

Out: RandomForest test_accuracy=0.9708
LogisticRegression test_accuracy=0.9708
GradientBoosting test_accuracy=0.9591

• Artifact 전송 방식의 차이

> 누가 저장소 자격 증명을 가지고 파일을 쓰는가?

- 서버 경유: `--serve-artifacts`와 `--artifacts-destination` 사용
- 직접 저장: `--no-serve-artifacts`와 `--default-artifact-root` 사용
- 서버 경유에서는 서버가 저장소에 접근할 권한을 보유
- 직접 저장에서는 학습 PC에도 저장소 접근 권한이 필요
- 기존 Experiment의 `artifact_location`은 설정 변경만으로 이사하지 않음

MLflow

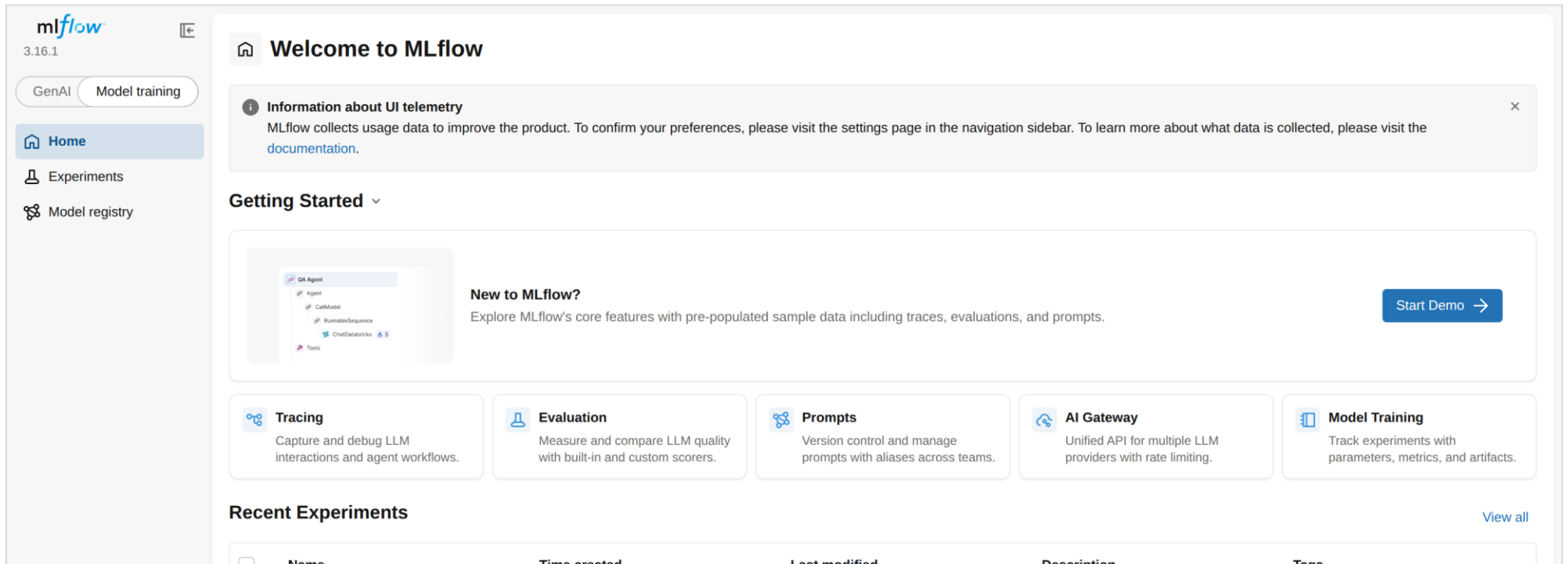
• MLflow UI로 실험 비교하기

> MLflow 홈 화면 살펴보기

> 왼쪽 메뉴의 'Model training'을 선택하면 우리가 배우는 실험 추적 화면으로 전환됩니다.

- MLflow는 GenAI(LLM 추적) 기능까지 함께 제공하는 통합 플랫폼으로 발전했습니다.

24



MLflow 3.x 홈 화면 (실습 환경 실제 캡처)

- MLflow UI로 실험 비교하기

- > Experiments 목록 화면
- > 실습 중 만든 Experiment들이 모두 이 목록에 쌓입니다.

 3.16.1

GenAI

Model training

Home

Experiments

Model registry

Experiments

Search experiments

Q

Tag

☐	Name	Time created	Last modified	Description	Tags
☐	breast-cancer-gridsearchcv	09/21/2026, 09:33:05 AM	09/21/2026, 09:33:05 AM	-	
☐	breast-cancer-tuning	09/21/2026, 09:33:03 AM	09/21/2026, 09:33:03 AM	-	
☐	breast-cancer-autolog	09/21/2026, 09:32:51 AM	09/21/2026, 09:32:51 AM	-	
☐	breast-cancer-basic-tracking	09/21/2026, 09:32:50 AM	09/21/2026, 09:32:50 AM	-	
☐	Default	09/21/2026, 09:32:50 AM	09/21/2026, 09:32:50 AM	-	

< Previous

Next >

25 / page

Create

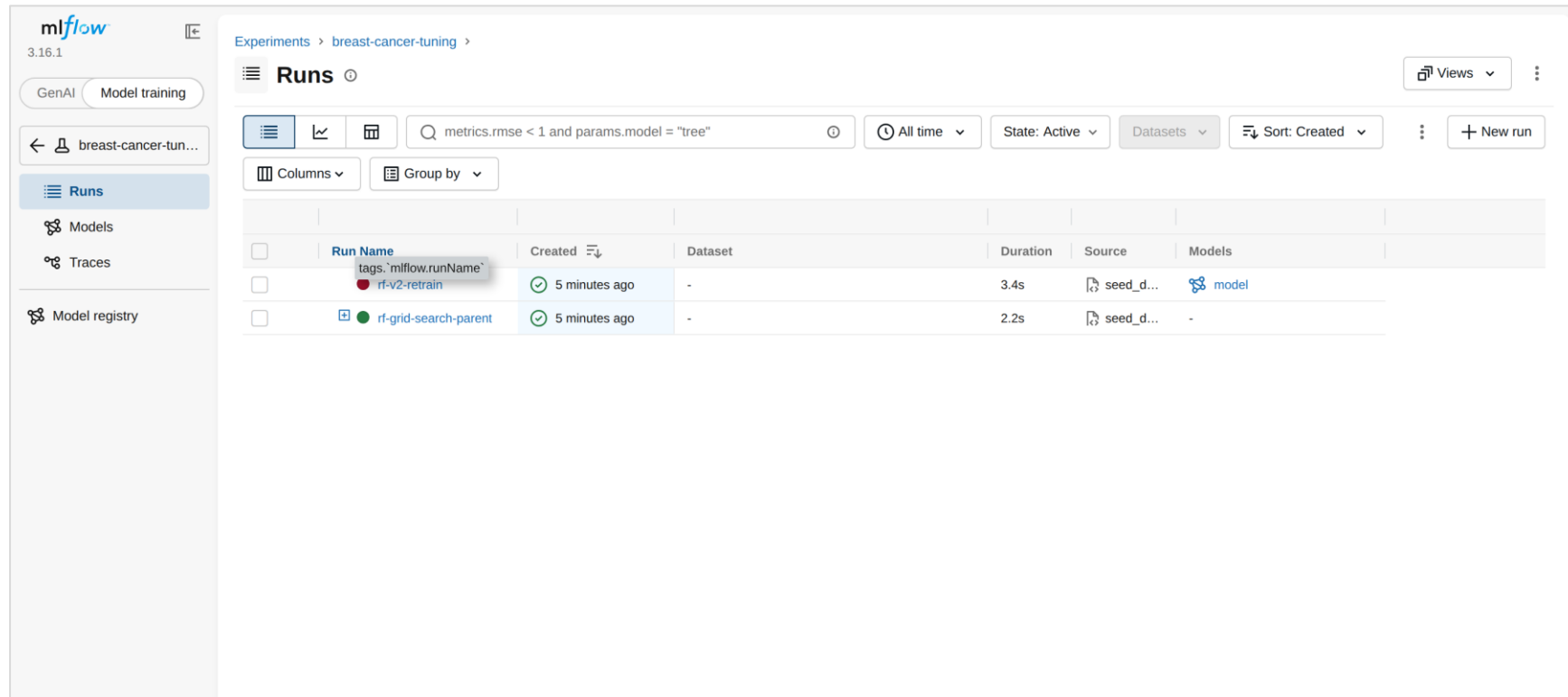
Compare

Delete

Experiments 목록

- MLflow UI로 실험 비교하기

- > Runs 목록과 검색
- > 검색창에 조건식을 넣으면 원하는 Run만 걸러볼 수 있습니다.
 - 예: `metrics.accuracy > 0.95 and params.max_depth = "5"`



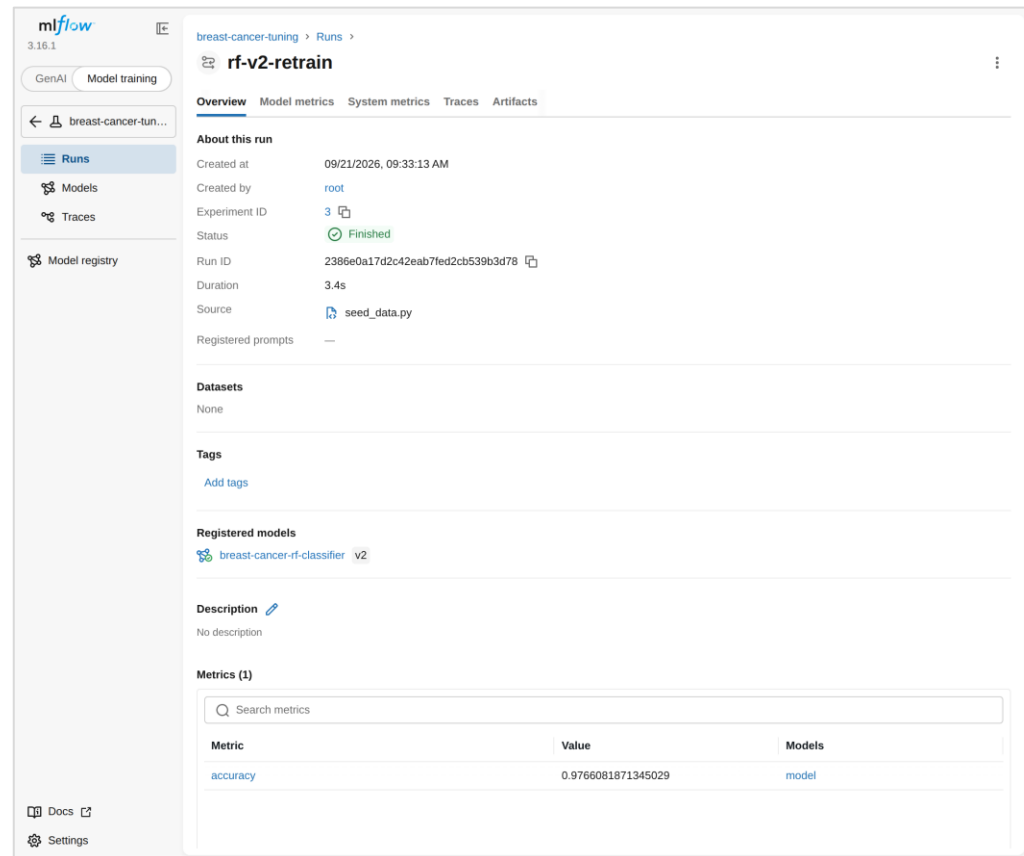
MLflow

- MLflow UI로 실험 비교하기

- > Run 상세 화면 보기

- > Overview 탭에서 파라미터·지표·등록된 모델까지 한 화면에서 확인할 수 있습니다.

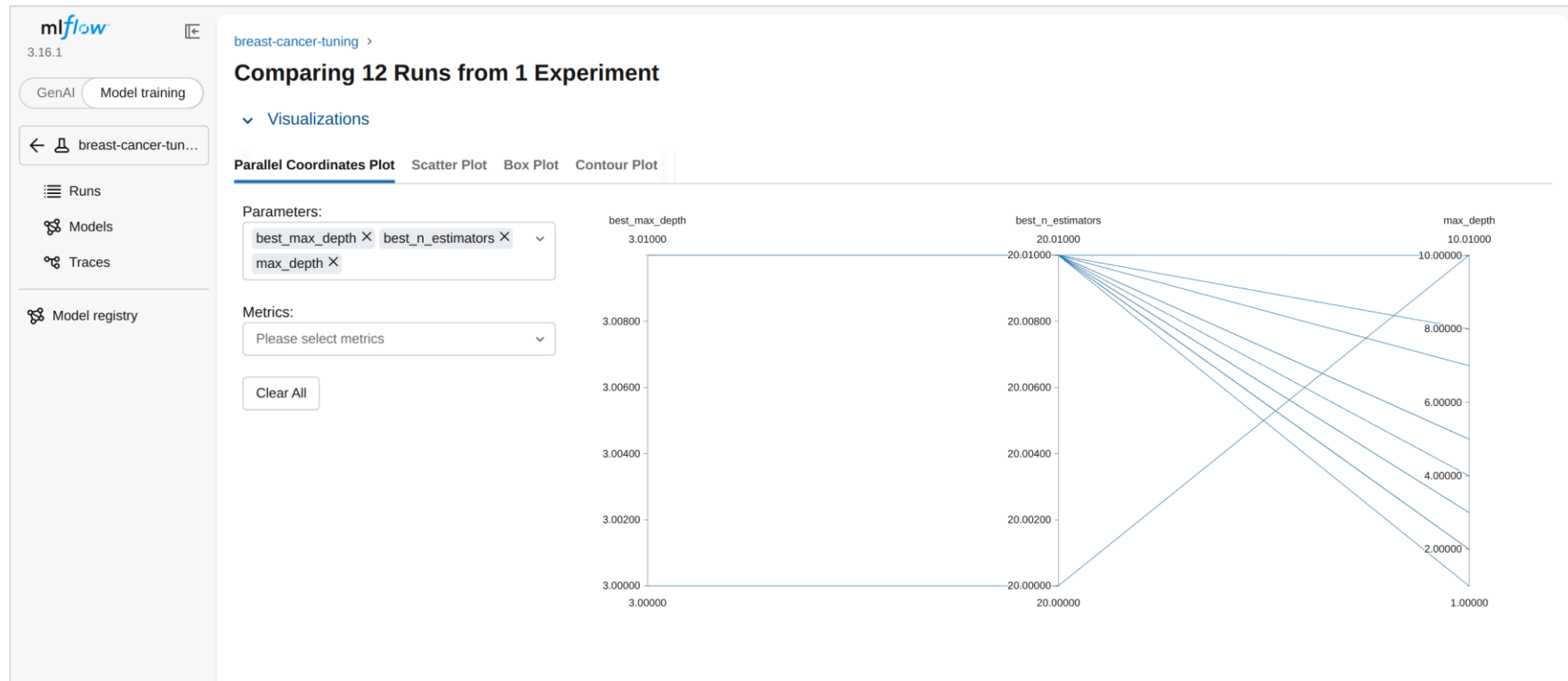
27



- MLflow UI로 실험 비교하기

- > 실습: 여러 실험 결과 비교해보기
- > 체크박스로 여러 Run을 선택한 뒤 Compare 버튼을 누르면 Parallel Coordinates Plot으로 비교할 수 있습니다.

28



Compare Runs — Parallel Coordinates Plot

- 원격 서버 연결과 점검

> 실제 서버 주소와 Experiment의 저장 위치 확인

```
import os
from mlflow import MlflowClient
uri = os.environ["MLFLOW_TRACKING_URI"]
client = MlflowClient(tracking_uri=uri)
experiment = client.get_experiment_by_name("wine-classification")
print("Tracking URI:", uri)
if experiment is not None:
    print("Artifacts:", experiment.artifact_location)
```

- 실습 문제와 평가 기준

- > Wine 다중 분류 · 점수보다 기록 과정에 집중

- scikit-learn 내장 Wine 데이터로 3개 클래스를 분류
- 학습용 약 60%, 검증용 약 20%, 최종 테스트용 약 20%
- 주 지표: macro F1, 보조 지표: accuracy
- 파라미터 선택은 검증 데이터로만 수행
- 최종 테스트는 후보 선택을 마친 뒤 한 번 평가

- 공통 코드 준비

> 동일한 Python 세션에서 순서대로 실행

```
import mlflow
import mlflow.sklearn
import numpy as np
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score
```

- 학습·검증·테스트 분리

> 분할을 고정해야 Run 간 비교가 가능하다

```
wine= load_wine()
X = wine.data
y = wine.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y,
                                                    random_state=42)
X_tr, X_val, y_tr, y_val = train_test_split(X_train, y_train, test_size=0.2,
                                            stratify=y_train, random_state=42)
print(len(X_tr), len(X_val), len(X_test))
```

- 전처리를 포함한 모델

> Pipeline 전체를 학습하고 나중에 함께 저장

```
def make_model(C=1.0):  
    return Pipeline([  
        ("scale", StandardScaler()),  
        ("model", LogisticRegression(  
            C=C, max_iter=2000, random_state=SEED))  
    ])  
  
pipe = make_model()  
pipe.fit(X_tr, y_tr)  
pred = pipe.predict(X_val)
```

- 첫 번째 Run

> Experiment 선택 후 학습 결과를 한 실행에 기록

```
mlflow.set_tracking_uri("http://127.0.0.1:5000")
exp = mlflow.set_experiment("wine-classification")
with mlflow.start_run(run_name="lr-baseline") as run:
    pipe = make_model(C=1.0)
    pipe.fit(X_train, y_train)
    pred = pipe.predict(X_val)
    mlflow.log_param("C", 1.0)
    mlflow.log_metric("val_f1_macro",
                      f1_score(y_val, pred, average="macro"))
    baseline_id = run.info.run_id
print(baseline_id)
```

with 블록을 정상적으로 벗어나면 Run이 FINISHED 상태로 종료됩니다.

- Parameter의 기록 규칙

> 한 Run 안에서 같은 키의 값은 변경하지 않는다

```
with mlflow.start_run(run_name="params-demo"):
    mlflow.log_params({
        "model_type": "logistic_regression",
        "C": 1.0, "seed": SEED,
        "split_id": "wine-seed42-v1"
    })
    # mlflow.log_param("C", 10.0) # 값 충돌

# C를 변경해 학습하려면 새로운 Run을 시작한다
```

Parameter는 문자열로 저장됩니다. 숫자처럼 보이는 값도 검색에서는 문자열입니다.

- Metric과 Tag

> 수치 평가 결과와 실행 설명을 함께 기록

```
with mlflow.start_run(run_name="metrics-tags"):
    pipe = make_model().fit(X_train, y_train)
    pred = pipe.predict(X_val)
    mlflow.log_metrics({
        "val_accuracy": accuracy_score(y_val, pred),
        "val_f1_macro": f1_score(y_val, pred, average="macro")
    })
    mlflow.set_tags({"purpose": "baseline", "owner": "student"})
    mlflow.set_tag("decision", "비교 기준 모델")
```

train_, val_, test_ 접두사를 구분합니다. Tag는 나중에 수정할 수 있습니다.

- Artifact로 남길 증거

- > 수치 하나로 설명할 수 없는 결과 파일

- 혼동행렬 그림: 어떤 클래스를 서로 혼동했는가?
- JSON 설정: 어떤 평가 규칙과 데이터 버전을 사용했는가?
- 분할 인덱스: 어떤 행을 학습·검증·테스트에 사용했는가?
- 코드·패키지 목록: 무엇을 실행했고 어떤 환경이었는가?
- 원본 데이터 저장 여부는 크기·권한·보존 규칙을 따로 결정

Artifact 저장은 파일 보관입니다. 파일 내용의 정확성은 별도로 점검해야 합니다.

- 설정과 설명 저장

> 앞에서 만든 baseline Run을 의도적으로 재개

```
with mlflow.start_run(run_id=baseline_id):  
    mlflow.log_dict({  
        "dataset": "sklearn-wine",  
        "split_id": "wine-seed42-v1",  
        "primary_metric": "val_f1_macro"  
    }, "config/evaluation.json")  
    mlflow.log_text(  
        "StandardScaler와 LogisticRegression 기준 모델",  
        "notes/baseline.txt")
```

새로운 학습은 새 Run으로, 기존 실행의 설명 보완은 run_id를 지정해 재개합니다.

- 혼동행렬 저장

> 현재 예측 결과를 그림 Artifact로 기록

```
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay
pipe = make_model().fit(X_train, y_train)
pred = pipe.predict(X_val)
with mlflow.start_run(run_id=baseline_id):
    fig, ax = plt.subplots(figsize=(5, 4))
    ConfusionMatrixDisplay.from_predictions(
        y_val, pred, ax=ax)
    mlflow.log_figure(fig, "plots/confusion_matrix.png")
    plt.close(fig)
```

그림과 metric이 같은 데이터·같은 모델에서 나왔는지 확인합니다.

- 데이터 계보 기록

> log_input은 데이터 자체의 완전한 백업이 아니다

```
train_df = X_train.assign(target=y_train)
train_ds = mlflow.data.from_pandas(
    train_df, name="wine-train", targets="target")
with mlflow.start_run(run_id=baseline_id):
    mlflow.log_input(train_ds, context="training")
    mlflow.set_tag("data_version", "sklearn-wine-v1")
```

Dataset의 이름·digest·schema와 별도로 원본 데이터 위치와 버전을 보관합니다.

- 재현성에 필요한 기록

> seed와 함께 분할·패키지·코드도 보관

```
import subprocess, sys
versions = subprocess.check_output(
    [sys.executable, "-m", "pip", "freeze"], text=True)
with mlflow.start_run(run_id=baseline_id):
    mlflow.log_text(versions, "env/requirements.txt")
    mlflow.log_dict({
        "train": X_train.index.tolist(),
        "val": X_val.index.tolist(),
        "test": X_test.index.tolist()
    }, "data/split_indices.json")
```

추가 권장: 코드 파일 또는 Git commit, 데이터 버전, 실행 명령, OS·Python 버전.

- Metric의 step과 이력

> 같은 키로 반복 평가의 순서를 기록

```
from mlflow import MlflowClient
with mlflow.start_run(run_name="step-demo") as run:
    for step, score in enumerate([0.70, 0.78, 0.76]):
        mlflow.log_metric("demo_score", score, step=step)
    history_id = run.info.run_id
client = MlflowClient()
history = client.get_metric_history(history_id, "demo_score")
print([(m.step, m.value) for m in history])
```

위 점수는 이력 설명용 가상 값입니다. 마지막 값과 최고 값은 다를 수 있습니다.

- Run 상태와 예외

> 실패도 비교 가능한 기록으로 남긴다

```
try:
    with mlflow.start_run(run_name="failure-demo"):
        mlflow.set_tag("purpose", "error-demo")
        raise ValueError("교육용 의도적 오류")
except ValueError as error:
    print(error)

print(mlflow.active_run()) # None
```

예외를 with 내부에서 삼키면 정상 종료로 보일 수 있습니다. 실패 원인을 기록하세요.

• 재현성 점검

> 동료가 이 실행을 다시 수행할 수 있는가?

- 데이터 버전과 분할 인덱스가 있다
- 전처리를 포함한 학습 코드와 파라미터가 있다
- Python·패키지 버전과 실행 명령이 있다
- 검증 지표의 정의와 계산 대상이 명확하다
- 다음 단계: 같은 예측을 만드는 모델을 함께 저장

질문: random_state=42만 남겼다면 무엇이 부족할까요?

• MLflow Model의 구성

> 예측 객체를 다시 쓰기 위한 저장 형식

- 모델 객체: Pipeline의 학습된 전처리와 분류기
- MLmodel: flavor와 signature 등의 메타데이터
- 환경 파일: requirements.txt, python_env.yaml 등
- input_example: 입력 형태를 보여주는 작은 예시
- log_model: Tracking에 저장 / save_model: 로컬 경로에 저장

Model을 로그하는 것과 Registry에 등록하는 것은 서로 다른 작업입니다.

- Signature와 입력 예시

> 모델이 받는 열 이름과 자료형을 명시

```
from mlflow.models import infer_signature
pipe = make_model().fit(X_train, y_train)
example = X_train.head(3)
signature = infer_signature(
    example, pipe.predict(example))
print(signature)
```

Signature는 입력 계약을 돕지만 데이터 분포·품질까지 보장하지는 않습니다.

- Pipeline 모델 저장

> MLflow 3의 name과 반환된 model_uri 사용

```
with mlflow.start_run(run_name="lr-model") as run:
    mlflow.log_params({"C": 1.0, "seed": SEED})
    pipe = make_model().fit(X_train, y_train)
    info = mlflow.sklearn.log_model(
        sk_model=pipe, name="wine-pipeline",
        signature=signature, input_example=example)
    mlflow.set_tag("saved_model_uri", info.model_uri)
    model_run_id = run.info.run_id
    print(info.model_uri)
```

MLflow 3의 Logged Model에는 model_id가 있습니다. 반환된 URI를 그대로 보관합니다.

- 모델 불러오기

> 저장 전후 예측이 같은지 확인

```
loaded = mlflow.sklearn.load_model(info.model_uri)
before = pipe.predict(X_val)
after = loaded.predict(X_val)
np.testing.assert_array_equal(before, after)
print("예측 일치:", len(after), "개")

prob = loaded.predict_proba(X_val)
print(prob.shape)
```

sklearn flavor로 불러오면 Pipeline의 predict와 predict_proba를 사용할 수 있습니다.

- pyfunc와 sklearn flavor

> 호출 방식이 필요한 기능에 맞는지 확인

```
generic = mlflow.pyfunc.load_model(info.model_uri)
labels = generic.predict(X_val.head(3))
print(labels)
```

```
# 일반 sklearn 기록의 pyfunc는 기본적으로 predict 결과 사용
# 확률이 필요하다면 sklearn flavor로 로드한 객체 사용
proba = loaded.predict_proba(X_val.head(3))
```

pyfunc의 predict가 분류 확률을 반환한다고 가정하지 않습니다.

- MLmodel 파일 읽기

- > 저장 형식을 설명하는 YAML 메타데이터

- flavors: 모델을 어떤 로더와 예측 인터페이스로 읽을지 설명
- sklearn: 저장 파일과 scikit-learn 버전 등의 정보
- python_function: 범용 predict를 제공하는 로딩 정보
- signature: 입력과 출력의 열 또는 텐서 규격
- 환경 파일과 입력 예시는 MLmodel의 참조 정보와 함께 확인

MLmodel 안의 메타데이터와 실제 학습된 모델 파일은 구분합니다.

- 저장 모델의 내부 확인

> 원본 30장의 info.model_uri로 다운로드

51

```
from pathlib import Path
folder = mlflow.artifacts.download_artifacts(
    artifact_uri=info.model_uri)
print([p.name for p in Path(folder).iterdir()])
text = (Path(folder) / "MLmodel").read_text("utf-8")
print(text)
meta = mlflow.models.get_model_info(info.model_uri)
print(meta.flavors.keys())
print(meta.signature)
```

확인할 것: MLmodel, 모델 파일, requirements.txt, 입력 예시 파일.

- Model API의 역할

- > 저장, 기록, 로딩, 예측의 경계를 구분

- `save_model(path=...)`: 로컬 모델 폴더를 생성
- `log_model(name=...)`: Tracking에 기록하고 `ModelInfo`를 반환
- `load_model(uri)`: 현재 Python 환경으로 모델을 로드
- `get_model_info(uri)`: flavor와 signature 등의 메타데이터 조회
- `models predict / serve`: 파일 일괄 예측 또는 HTTP 예측 제공

`load_model`은 필요한 패키지를 자동 설치하지 않습니다.

- Model URI의 종류

- > 문자열이 가리키는 대상을 구분한다

- models:/<model_id> : MLflow 3의 Logged Model
- runs:/<run_id>/<artifact_path> : 실행 안의 모델 경로
- models:/wine-classifier/1 : Registry의 특정 버전
- models:/wine-classifier@champion : alias가 가리키는 버전
- 이번 수업: log_model의 info.model_uri를 우선 사용

run_id, model_id, Registry version은 서로 다른 식별자입니다.

- 입력 규격 오류 확인

> 열 하나를 빠뜨린 입력으로 로드 모델 호출

```
bad_input = X_val.head(3).drop(columns=[X_val.columns[0]])
try:
    generic.predict(bad_input)
except Exception as error:
    print(type(error).__name__)
    print(str(error)[:250])

# 원래 열을 유지한 DataFrame은 정상 예측
generic.predict(X_val.head(3))
```

오류를 숨기기보다 누락 열·자료형·전처리 순서를 확인합니다.

• Custom PythonModel의 용도

> 예측 결과에 확률 정보나 업무 규칙을 더하는 경우

- 표준 sklearn flavor로 표현하기 어려운 입출력을 직접 정의
- 예제: Wine 예측에 label과 confidence 열을 함께 반환
- load_context: 함께 저장한 Pipeline을 한 번 로드
- predict: 입력 DataFrame을 받아 결과 DataFrame을 반환
- 이번 예제는 Python 파일을 저장하는 Models from Code 방식

confidence는 최대 예측 확률이며 정답일 확률을 보장하지는 않습니다.

- Custom Model 정의

> wine_pyfunc.py 파일로 저장

```
import mlflow
import pandas as pd
class WineOutput(mlflow.pyfunc.PythonModel):
    def load_context(self, context):
        self.pipe = mlflow.sklearn.load_model(
            context.artifacts["pipeline"])
    def predict(self, context, model_input, params=None):
        return pd.DataFrame({
            "label": self.pipe.predict(model_input),
            "confidence": self.pipe.predict_proba(model_input).max(1)})
mlflow.models.set_model(WineOutput())
```

Pipeline 위치는 개인 PC 경로 대신 context.artifacts에서 받습니다.

• Custom Model 기록

> 코드와 의존 Pipeline을 함께 저장

```
import tempfile
from pathlib import Path
with tempfile.TemporaryDirectory() as temp:
    base_path = str(Path(temp) / "pipeline")
    mlflow.sklearn.save_model(pipe, base_path)
    with mlflow.start_run(run_name="custom-output"):
        custom_info = mlflow.pyfunc.log_model(
            name="wine-output", python_model="wine_pyfunc.py",
            artifacts={"pipeline": base_path},
            input_example=X_train.head(3))
```

input_example로 추론한 signature도 저장 결과에서 직접 확인합니다.

- Custom Model 재로딩

> 새로 로드한 모델의 출력 계약 검사

```
custom = mlflow.pyfunc.load_model(custom_info.model_uri)
output = custom.predict(X_val)
print(output.head())
assert list(output.columns) == ["label", "confidence"]
np.testing.assert_array_equal(
    output["label"].to_numpy(), pipe.predict(X_val))
assert output["confidence"].between(0, 1).all()
print(mlflow.models.get_model_info(
    custom_info.model_uri).signature)
```

label 일치, confidence 범위, 출력 열 이름을 모두 확인합니다.

• Custom Flavor의 범위

> 반복 사용하는 모델 형식을 모듈로 표준화

- Custom PythonModel: 범용 pyfunc의 예측 동작을 정의
- Custom Flavor: 전용 save_model, load_model 등의 규약을 정의
- 예제: sklearn 형식을 재사용하고 wine_pipeline 정보를 추가
- 별도 flavor 메타데이터에 업무용 schema_version을 기록
- pyfunc 호환성은 기존 sklearn 저장 기능을 통해 유지

새 직렬화 엔진 구현보다 저장·로딩 규약의 책임을 이해하는 예제입니다.

- Custom Flavor 저장 함수

> wine_flavor.py의 save_model

```
from pathlib import Path
import mlflow
from mlflow.models import Model
FLAVOR = "wine_pipeline"
def save_model(model, path, mlflow_model=None, **kwargs):
    meta = mlflow_model if mlflow_model is not None else Model()
    mlflow.sklearn.save_model(
        model, path, mlflow_model=meta, **kwargs)
    meta.add_flavor(FLAVOR, schema_version=1)
    meta.save(str(Path(path) / "MLmodel"))
```

MLmodel에 전용 flavor를 추가하면서 sklearn과 python_function을 보존합니다.

- Custom Flavor 로딩 함수

> wine_flavor.py의 load_model

```
def load_model(uri):  
    folder = mlflow.artifacts.download_artifacts(  
        artifact_uri=uri)  
    meta = Model.load(str(Path(folder) / "MLmodel"))  
    config = meta.flavors.get(FLAVOR)  
    if config is None or config.get("schema_version") != 1:  
        raise ValueError("지원하지 않는 wine_pipeline 형식")  
    return mlflow.sklearn.load_model(folder)
```

전용 형식의 버전을 검사한 뒤 실제 복원은 sklearn flavor에 맡깁니다.

• Custom Flavor 확인

> 전용 로더와 pyfunc 로더의 결과 비교

```
import wine_flavor
from mlflow.models import Model
with tempfile.TemporaryDirectory() as temp:
    path = str(Path(temp) / "wine_model")
    wine_flavor.save_model(pipe, path)
    native = wine_flavor.load_model(path)
    generic = mlflow.pyfunc.load_model(path)
    np.testing.assert_array_equal(
        native.predict(X_val), generic.predict(X_val))
    print(Model.load(str(Path(path) / "MLmodel")).flavors.keys())
```

확인할 flavor: sklearn, python_function, wine_pipeline.

- 반복 실험의 비교 조건

- > 바꾸는 조건과 고정하는 조건을 분리

- 고정: 데이터 버전, 분할 인덱스, 평가 지표, seed
 - 변경: LogisticRegression의 C
 - 후보마다 새 Run과 새 Pipeline 생성
 - 각 Run에 조건·검증 점수·모델 URI 기록
 - 작은 데이터의 한 번의 점수 차이를 과장하지 않는다

테스트 점수로 정렬하지 않습니다. 후보 선택에는 `val_f1_macro`를 사용합니다.

- 반복 실험 함수 ①

> 각 후보의 파라미터와 지표 기록

```
def run_trial(C):  
    with mlflow.start_run(run_name=f"lr-C{C}") as run:  
        model = make_model(C).fit(X_train, y_train)  
        pred = model.predict(X_val)  
        mlflow.log_params({"C": C, "seed": SEED,  
                           "split_id": "wine-seed42-v1"})  
        mlflow.set_tag("purpose", "manual-sweep")  
        mlflow.log_metrics({  
            "val_accuracy": accuracy_score(y_val, pred),  
            "val_f1_macro": f1_score(y_val, pred, average="macro")  
        })
```

다음 장의 코드가 같은 함수의 with 블록 안에서 이어집니다.

• 반복 실험 함수 ②

> 모델 저장 후 후보 4개 실행

```
info = mlflow.sklearn.log_model(  
    model, name="wine-pipeline",  
    input_example=X_train.head(3),  
    signature=infer_signature(  
        X_train.head(3), model.predict(X_train.head(3))))  
mlflow.set_tag("saved_model_uri", info.model_uri)  
return run.info.run_id
```

```
trial_ids = [run_trial(C) for C in [0.0001, 0.001, 0.01, 1.0]]
```

맨 위 7줄은 38장의 함수에 이어 붙입니다. trial_ids는 이번 비교 대상입니다.

• UI에서 실행 비교

> 조건과 지표를 함께 놓고 읽는다

- wine-classification에서 purpose>manual-sweep 실행 찾기
- 이번에 실행한 4개 Run을 선택하고 Compare 열기
- C, seed, split_id와 val_f1_macro를 함께 확인
- 검증 점수가 같다면 단순성·비용 같은 보조 기준 검토
- 후보 Run의 모델 URI와 Artifact까지 확인

UI의 위치·문구는 버전에 따라 달라질 수 있습니다. 수업은 3.3.2 기준입니다.

- search_runs로 조회

> 조건 검색 결과를 DataFrame으로 다룬다

```
runs = mlflow.search_runs(  
    experiment_ids=[exp.experiment_id],  
    filter_string="tags.purpose = 'manual-sweep' "  
                 "AND attributes.status = 'FINISHED'",  
    order_by=["metrics.val_f1_macro DESC"])  
print(runs[["run_id", "params.C", "metrics.val_f1_macro"]])
```

기본 반환값은 pandas DataFrame입니다. 기록이 없는 열의 값은 비어 있을 수 있습니다.

- 이번 후보에서 최종 선택

> 검증 점수와 동점 규칙을 명시

```
candidates = runs[runs["run_id"].isin(trial_ids)].copy()
candidates["C_num"] = candidates["params.C"].astype(float)
candidates = candidates.sort_values(
    ["metrics.val_f1_macro", "C_num"], ascending=[False, True])
assert not candidates.empty
best = candidates.iloc[0]
best_uri = best["tags.saved_model_uri"]
best_run_id = best["run_id"]
print(best_run_id, best_uri)
```

수업용 동점 규칙: macro F1이 같으면 더 작은 C. 업무 기준은 사전에 정합니다.

- 비교할 때 흔한 오해

- > 저장한 지표의 의미부터 확인

- training_score가 가장 높은 Run을 최종 모델로 고르지 않는다
- 다른 데이터 버전의 점수를 같은 조건의 순위로 해석하지 않는다
- 마지막 metric 값이 항상 최고 값인 것은 아니다
- 실패한 Run이나 지표가 없는 Run은 후보에서 제외
- 지표가 같을 때의 판단 기준을 결과 확인 전에 정한다

질문: val_f1이 같아도 어느 모델을 선택할지 설명할 수 있나요?

• Model Evaluation의 역할

> 선택 모델을 같은 데이터와 평가 규칙으로 진단

- 수동 log_metric은 이미 계산한 수치를 기록
- models.evaluate는 예측·지표 계산·평가 Artifact 생성을 수행
- 입력은 저장 모델 URI, 검증 데이터, 정답 열 이름
- 분류 지표의 average와 데이터 구분 접두사를 명시
- 이번 예제는 선택 모델의 검증 진단이며 최종 테스트는 보류

자동 평가가 데이터 누수나 잘못된 평가 설계를 해결하지는 않습니다.

- 선택 모델 자동 평가

> 기존 val_f1_macro와 같은 macro 평균 사용

```
eval_data = X_val.assign(target=y_val)
with mlflow.start_run(run_name="selected-model-evaluation"):
    mlflow.set_tag("selected_model_uri", best_uri)
    result = mlflow.models.evaluate(
        model=best_uri, data=eval_data, targets="target",
        model_type="classifier", evaluators="default",
        evaluator_config={"average": "macro",
                          "metric_prefix": "auto_val_",
                          "log_model_explainability": False})
    print(result.metrics)
```

설명가능성 계산은 이번 예제에서 끄고 지표와 평가 파일을 확인합니다.

• 자동 평가 결과 해석

> 지표 이름과 산출 조건을 함께 확인

```
selected = mlflow.sklearn.load_model(best_uri)
manual = f1_score(y_val, selected.predict(X_val), average="macro")
automatic = result.metrics["auto_val_f1_score"]
np.testing.assert_allclose(manual, automatic)
print("manual / automatic:", manual, automatic)
print("평가 파일:", list(result.artifacts))
for name, artifact in result.artifacts.items():
    print(name, artifact.uri)
```

같은 모델·같은 입력·같은 평균 방식이면 두 F1이 일치해야 합니다.

• 자동 기록과 수동 기록

> autolog가 줄여 주는 작업과 남는 책임

- autolog: estimator 파라미터, 학습 지표, 모델 등을 자동 기록
- 수동 기록: 검증 지표 이름, 업무 태그, 선택 근거를 명시
- 지원되는 scikit-learn 버전과 옵션을 확인
- 자동 기록은 이후 fit 호출에도 영향을 줄 수 있다
- 수동 실습으로 돌아갈 때 `autolog(disable=True)` 호출

자동 기록으로 남은 학습 지표가 검증 성능을 대신하지는 않습니다.

- scikit-learn autolog

> 검증 지표는 이름을 정해 직접 추가

```
mlflow.sklearn.autolog(log_input_examples=True)
with mlflow.start_run(run_name="autolog-demo"):
    auto_model = make_model().fit(X_train, y_train)
    val_pred = auto_model.predict(X_val)
    mlflow.log_metric("val_f1_macro",
                      f1_score(y_val, val_pred, average="macro"))
    mlflow.set_tag("purpose", "autolog-demo")
mlflow.sklearn.autolog(disable=True)
```

UI에서 자동으로 생긴 항목과 직접 추가한 항목을 나누어 확인합니다.

- GridSearchCV와 자동 기록

> 하이퍼파라미터 탐색 결과도 추적

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold
cv = StratifiedKFold(3, shuffle=True, random_state=SEED)
mlflow.sklearn.autolog(max_tuning_runs=4)
with mlflow.start_run(run_name="grid-search"):
    grid = GridSearchCV(make_model(),
                        {"model__C": [0.0001, 0.001, 0.01, 1.0]},
                        cv=cv, scoring="f1_macro", n_jobs=1)
    grid.fit(X_train, y_train)
    mlflow.log_metric("val_f1_macro", f1_score(
        y_val, grid.predict(X_val), average="macro"))
mlflow.sklearn.autolog(disable=True)
```

model__C는 Pipeline 안 model 단계의 C입니다. 테스트는 탐색에 사용하지 않습니다.

• 중첩 Run의 역할

> 탐색 전체와 개별 후보의 기록을 연결

- Parent Run: 탐색의 목적, 공통 조건, 최종 선택 요약
- Child Run: 후보 하나 또는 fold 하나의 결과
- nested=True로 현재 Run 아래에 자식 Run 생성
- 자식은 mlflow.parentRunId 태그로 부모와 연결
- 자동 기록과 수동 중첩 구조를 섞을 때 중복 여부 확인

평면 Run으로 충분하다면 중첩 구조를 억지로 만들 필요는 없습니다.

- 수동 중첩 Run 예제

> CV 평균과 표준편차를 후보별 기록

```
from sklearn.model_selection import cross_val_score
with mlflow.start_run(run_name="manual-cv"):
    for C in [0.1, 1.0, 10.0]:
        with mlflow.start_run(run_name=f"C{C}", nested=True):
            scores = cross_val_score(
                make_model(C), X_train, y_train,
                cv=cv, scoring="f1_macro")
            mlflow.log_param("C", C)
            mlflow.log_metrics({"cv_f1_mean": scores.mean(),
                               "cv_f1_std": scores.std()})
```

autolog를 끈 상태로 실행합니다. CV 수행만으로 최종 모델이 저장되지는 않습니다.

- Run과 모델 기록의 연결

> MLflow 3에서 모델·데이터 단위로 지표 연결

```
val_ds = mlflow.data.from_pandas(
    X_val.assign(target=y_val), name="wine-val", targets="target")
with mlflow.start_run(run_name="linked-metric"):
    model = make_model().fit(X_train, y_train)
    model_info = mlflow.sklearn.log_model(
        model, name="linked-model", input_example=X_train.head(3))
    score = f1_score(y_val, model.predict(X_val), average="macro")
    mlflow.log_metric("val_f1_macro", score,
                      model_id=model_info.model_id, dataset=val_ds)
```

Run 검색과 Logged Model 검색은 대상과 API가 다릅니다.

• 탐색 결과 정리

> 어떤 점수로 무엇을 선택했는지 기록

- CV 평균: 학습 데이터 안에서 후보를 비교한 결과
- CV 표준편차: fold별 점수의 변동 요약
- Holdout 검증: 별도 검증 데이터에 대한 평가
- 최종 테스트: 선택 완료 후 한 번 확인할 성능
- Run 이름만 보고 지표의 계산 대상을 추측하지 않는다

다음 차시 Registry에는 5차시에서 선택한 best_uri를 등록합니다.

• MLflow Projects의 역할

> 학습 코드를 파라미터가 있는 실행 단위로 공유

- MLproject 파일에 실행 진입점과 파라미터를 선언
- 학습 코드는 train.py, 공통 함수는 common.py에 분리
- MLflow Model은 저장된 예측 객체를 재사용하는 형식
- MLflow Project는 코드를 다시 실행하기 위한 정의
- --env-manager local은 현재 환경을 쓰므로 환경 준비가 선행

실습은 보충 ZIP의 project 폴더에서 실행합니다.

- MLproject 파일과 학습 진입점

> entry_points의 main이 실행할 명령 정의

```
# MLproject 파일
name: wine-workshop
entry_points:
  main:
    parameters:
      C: {type: float, default: 1.0}
    command: "python train.py --C {C}"

# train.py는 argparse로 C를 받고 start_run() 안에서 학습
# 전체 코드는 보충 ZIP의 project/train.py 참고
```

MLproject는 확장자 없는 파일입니다. YAML 들여쓰기를 유지합니다.

• Project 실행과 Run 확인

> 같은 코드에 서로 다른 C를 전달

```
# PowerShell: 가상환경을 활성화한 뒤 project 폴더에서 실행
$env:MLFLOW_TRACKING_URI = "http://127.0.0.1:5000"
mlflow run . --env-manager local `
  --experiment-name wine-projects -P C=0.01
mlflow run . --env-manager local `
  --experiment-name wine-projects -P C=1.0

# UI에서 wine-projects의 두 Run을 비교
# C, val_f1_macro, 모델 URI 확인
```

local 모드는 의존성을 설치하지 않습니다. 학습 전 requirements를 설치합니다.

• Model Registry의 역할

> 저장한 모델의 버전과 사용 대상을 관리

- Registered Model: wine-classifier 같은 모델 이름
- Model Version: 등록할 때 생성되는 버전 번호
- Alias: champion처럼 특정 버전을 가리키는 이름
- Version Tag: 검토 결과나 용도에 대한 메타데이터
- 모델 등록과 alias 변경만으로 서비스 배포가 실행되지는 않는다

수업의 SQLite backend는 Registry를 위한 DB 기반 저장소 역할도 합니다.

- 선택 모델 등록

> 5차시의 best_uri를 Registry에 추가

```
from mlflow import MlflowClient
client = MlflowClient()
registered_name = "wine-classifier"
version = mlflow.register_model(best_uri, registered_name)
client.set_model_version_tag(
    registered_name, version.version, "review", "classroom-approved")
print(registered_name, version.version)
```

반복 실행하면 새 버전이 추가됩니다. Run ID와 Registry 버전은 다릅니다.

- Alias로 모델 지정

> champion은 현재 지정한 버전을 가리킨다

```
client.set_registered_model_alias(  
    registered_name, "champion", version.version)  
alias_uri = f"models:{registered_name}@champion"  
champion = mlflow.sklearn.load_model(alias_uri)  
print(champion.predict(X_val.head(3)))  
  
resolved = client.get_model_version_by_alias(  
    registered_name, "champion")  
print("현재 버전:", resolved.version)
```

재현이 필요한 보고서는 alias와 함께 당시 버전 번호도 남깁니다.

• 모델 교체와 되돌리기

> 검토한 버전을 alias로 지정하는 절차

- 기존 champion의 버전 번호를 먼저 기록
- 새 버전의 검증 결과·입력 규격·재로딩 결과 확인
- 검토를 통과한 버전에 champion alias 지정
- 문제가 있으면 기존 버전으로 alias를 되돌림
- 기존 Staging / Production stage 예제는 alias 방식과 구분

Registry stage는 deprecated 상태입니다. 신규 수업 예제는 alias를 사용합니다.

- Fluent API와 MlflowClient

- > 활성 Run을 쓰는 방식과 run_id를 직접 지정하는 방식

- mlflow.start_run과 log_metric: 현재 활성 Run을 중심으로 기록
 - MlflowClient: run_id 등을 직접 지정해 생성·기록·조회
 - Client의 create_run은 fluent의 활성 Run을 설정하지 않음
 - Client로 만든 Run은 set_terminated로 종료 상태를 관리
 - 여러 실행의 조회, 관리 도구, 자동화 코드에서 Client를 활용

기존 Registry 코드의 Client를 Run 관리까지 확장해 봅니다.

- Client로 Run 생성과 조회

> 활성 Run 없이 ID를 명시해 기록

```
client = MlflowClient()
run = client.create_run(exp.experiment_id)
rid = run.info.run_id
try:
    client.log_param(rid, "purpose", "client-demo")
    client.set_tag(rid, "owner", "student")
except Exception:
    client.set_terminated(rid, status="FAILED")
    raise
else:
    client.set_terminated(rid, status="FINISHED")
```

확인: `client.get_run(rid).data.params` / `client.get_run(rid).info.status`

- 팀에서 사용하는 Tracking

- > 공유 주소와 저장소·접근 정책을 함께 설계

- 개인 PC: localhost + SQLite + 로컬 artifacts
- 팀 서버: 공유 Tracking URI + DB + 공유 Artifact Store
- 백업은 메타데이터 DB와 실제 Artifact 파일을 함께 수행
- 인증·네트워크 경계·저장소 권한을 별도로 구성
- 팀 규칙: 실험 이름, 태그, 지표, 데이터 버전, 보존 기간

수업용 localhost 명령을 그대로 외부에 공개하는 운영 구성으로 사용하지 않습니다.

- 문제가 생겼을 때 확인 순서

- > 증상에서 연결·Run·파일을 차례로 확인

- Run이 안 보임: `get_tracking_uri()`와 UI 주소 확인
- Parameter 충돌: 같은 Run에 다른 값으로 기록했는지 확인
- Run already active: `active_run()`과 with 블록 종료 확인
- 모델 로드 실패: URI, 서버 연결, Artifact, 패키지 버전 확인
- 입력 오류: signature의 열 이름·자료형과 실제 입력 비교

Experiment를 삭제하기 전에 연결 주소와 실행 ID부터 확인합니다.

• Registry 이해 확인

> 다섯 문장 중 올바른 설명 찾기

- ① log_model을 호출하면 반드시 Registry 버전도 생성된다
- ② alias가 가리키는 모델 버전은 바뀔 수 있다
- ③ Registry 버전 번호는 Run ID와 같은 값이다
- ④ 고정 버전 URI는 특정 등록 버전을 가리킨다
- ⑤ alias 변경만으로 모든 실행 중인 서비스가 자동 갱신된다

짜과 답을 비교하고 이유를 설명합니다.

- 자주 사용하는 MLflow CLI

- > 기능별 명령과 도움말

- mlflow server: Tracking UI와 기록 API 서버 실행
- mlflow experiments search: Experiment 목록 확인
- mlflow runs list --experiment-id ID: 실행 목록 조회
- mlflow models predict / serve: 일괄 예측 / 예측 서버
- mlflow run: MLflow Project 실행

실행 전 mlflow --version, 세부 옵션은 각 명령 뒤 --help로 확인합니다.

- CLI로 일괄 예측

> CSV 입력을 저장 모델로 예측

```
# Python: 원본 42장 실행 후
X_val.head(3).to_csv("input.csv", index=False)
print(best_uri) # 아래 MODEL_URI 자리에 붙여 넣기

# PowerShell: 가상환경과 Tracking URI 설정 후
mlflow models predict -m "MODEL_URI" `
  -i input.csv -t csv -o predictions.json `
  --env-manager local
Get-Content predictions.json
```

MODEL_URI는 예시 자리표시자입니다. models:/로 시작하는 실제 URI를 사용합니다.

- CLI로 예측 서버 실행

> Tracking Server와 별도의 포트 사용

```
# PowerShell: MODEL_URI를 실제 저장 모델 URI로 변경
mlflow models serve -m "MODEL_URI" `
  --host 127.0.0.1 --port 5001 --env-manager local
```

```
# 별도 Python 세션
```

```
import pandas as pd, requests
payload = {"dataframe_split": pd.read_csv("input.csv").to_dict("split")}
response = requests.post(
    "http://127.0.0.1:5001/invocations", json=payload, timeout=30)
response.raise_for_status()
print(response.json())
```

보충 requirements를 먼저 적용합니다. 5000번은 기록, 5001번은 예측입니다.